

DEAD BY DAYLIGHT

RPG - WEB EDITION

DOCUMENTACIÓN TÉCNICA Y MANUAL DE DESPLIEGUE AWS
EXCELENCIA ACADÉMICA - CALIFICACIÓN OBJETIVO: 10

AUTOR / DESARROLLADOR:

Noelia Cantador & Luis Lázaró

Proyecto Intermodular - Ciclo Superior DAM/DAW

STACK TECNOLÓGICO:

- Frontend: Next.js 16 (React / Tailwind CSS 4 / Zustand)
- Backend: Spring Boot 3.2 (Java 17 / JPA Hibernate / WebSockets)
- Base de Datos: MariaDB (Persistencia Híbrida JDBC-Transaccional)
- Calidad y Pruebas: JUnit 5 & Selenium WebDriver E2E Autómata
- Cloud: AWS (EC2 / RDS / VPC / Nginx Reverse Proxy / SSL Let's Encrypt)

Mayo, 2026 - Versión 2.0 Estable

CAPÍTULO 1: JUSTIFICACIÓN Y STACK TECNOLÓGICO

1.1 JUSTIFICACIÓN DEL PROYECTO

El proyecto Dead by Daylight RPG - Web Edition nace de la necesidad de fusionar dos mundos fascinantes: el diseño conceptual del clásico multijugador de terror asimétrico y la robustez del desarrollo de aplicaciones web full-stack empresariales. Se ha planteado como un RPG táctico por turnos donde la lógica procedimental orientada a objetos (POO) del backend se comunica de forma activa con una interfaz rica en micro-animaciones y con un rendimiento optimizado al milisegundo en el frontend.

El objetivo principal es lograr una integración limpia donde la consistencia de datos, la seguridad en la autenticación y la cobertura de pruebas de interfaz automáticas formen un sistema maduro, listo para producción.

1.2 STACK TECNOLÓGICO IMPLEMENTADO

Para garantizar los estándares más altos de robustez, escalabilidad y mantenibilidad, la arquitectura del sistema se ha estructurado en un modelo desacoplado de monorepo con las siguientes capas:

1. CAPA DE PRESENTACIÓN (FRONTEND): Next.js 16 (React) aprovechando el App Router para el enrutado optimizado en el servidor, Tailwind CSS 4 para un diseño estético premium con colores personalizados (inspirados en la niebla y la sangre del juego), y Zustand para la gestión de estados globales ligera, fluida y sin re-renderizados innecesarios.
2. CAPA DE NEGOCIO (BACKEND): Spring Boot 3.2 en Java 17. Se encarga de procesar los combates, validar los movimientos, gestionar las sesiones y emitir WebSockets para la comunicación reactiva en tiempo real durante la prueba.
3. CAPA DE DATOS (PERSISTENCIA HÍBRIDA): MariaDB 10.11 administrada. Combina la agilidad de JPA/Hibernate para el control de entidades simples (usuarios, sesiones) con el control total y milimétrico de JDBC en transacciones puras SQL para el guardado completo de las ranuras, garantizando rollbacks automáticos e integridad total ante excepciones.
4. CAPA DE PRUEBAS DE CALIDAD: Suite automatizada de JUnit 5 para el testeo unitario del motor de combate físico y Selenium WebDriver para simular y verificar el comportamiento real del juego con 5 usuarios simultáneos registrándose y jugando.

CAPÍTULO 2: CASOS DE USO Y FUNCIONALIDADES DEL RPG

2.1 CASOS DE USO PRINCIPALES DEL SISTEMA

El comportamiento dinámico del RPG se articula a través de cinco flujos de trabajo clave que conectan la base de datos con las acciones interactivas del jugador:

CASO DE USO 1: REGISTRO Y AUTENTICACIÓN SEGURA (HTTPONLY JWT)

El usuario introduce sus credenciales en la interfaz. El backend valida la petición y encripta la clave mediante BCrypt. Tras validarla, Spring Security genera un Token JWT firmado y lo inyecta directamente dentro de una cookie HTTP-Only llamada 'jwt_token'. Al ser HTTP-Only, JavaScript no puede acceder a ella, bloqueando de raíz cualquier ataque de robo de identidad (XSS).

CASO DE USO 2: GESTIÓN DE RANURAS DE GUARDADO MULTI-USUARIO

Cada usuario registrado tiene acceso exclusivo a exactamente 3 ranuras independientes de guardado en el dashboard. El sistema lee y representa de forma dinámica los detalles de cada ranura (modo, ronda, supervivientes/asesinos vivos). Si está vacía, permite instanciar una nueva partida. Las acciones sobre una ranura no afectan bajo ninguna circunstancia a los slots de otros usuarios.

CASO DE USO 3: COMBATE POR TURNOS (MANUAL / AUTOMÁTICO) CON WEBSOCKETS

El jugador selecciona sus personajes y comienza el combate (The Trial). El motor calcula el daño basándose en la precisión de las armas y los modificadores de las Perks. En modo Manual, el jugador decide la acción de cada personaje. En modo Automático, el motor corre ciclos autónomos de batalla y los actualiza de forma reactiva y en tiempo real a través de WebSockets en el navegador.

CASO DE USO 4: DESBLOQUEO AUTOMÁTICO DE LOGROS

A lo largo de las partidas, el sistema audita las condiciones de juego (ej. ganar sin recibir daño, asestar golpes críticos, jugar 1, 5 o 10 partidas). Al cumplirse los requisitos, el backend inserta de manera atómica el registro correspondiente en la tabla USUARIOS_LOGROS usando la instrucción INSERT IGNORE para evitar duplicidades.

CASO DE USO 5: HISTÓRICO Y ESTADÍSTICAS GLOBALES

Al finalizar una partida con la victoria de los supervivientes o la dominación del asesino, el sistema escribe los resultados finales en la tabla global de HISTORICO_PARTIDAS e inserta en lote (batch processing) el estado final de todos los personajes en HISTORICO_PERSONAJES. Esto nutre el salón de la fama (Hall of Fame) de la aplicación.

CAPÍTULO 3: DISEÑO DE BASE DE DATOS Y MAPEO RELACIONAL

3.1 DIAGRAMA ENTIDAD-RELACIÓN Y LÓGICA ESTRUCTURAL

El diseño de base de datos se ha concebido para soportar un modelo jerárquico complejo de persistencia de partidas en curso sin comprometer el rendimiento. La relación cardinal se define como:

- Un USUARIO tiene N RANURAS (Slots de guardado fijos del 1 al 3).
- Una RANURA aloja N PERSONAJE_PARTIDA (tanto supervivientes como asesinos que están combatiendo).
- Cada PERSONAJE_PARTIDA posee 0 o 1 ARMA_EQUIPADA, N PERK_EQUIPADA y N ESTADO_ACTIVO (efectos de estado alterado como hemorragia, regeneración o locura).
- El borrado de una partida o ranura se propaga de manera automática a cascada a nivel de base de datos (`ON DELETE CASCADE`) sobre los personajes, armas, perks y estados asociados, evitando dejar registros huérfanos y garantizando una base de datos limpia.
- Los usuarios tienen una relación N:M con la tabla maestra de LOGROS a través de la tabla intermedia USUARIOS_LOGROS.

3.2 PASO A TABLAS Y DICCIONARIO DE DATOS

Tabla: RANURA (Slots de Partida)

Campo	Tipo de Datos	Restricción	Descripción
id_ranura	INT	PRIMARY KEY	Identificador de ranura (1, 2 o 3)
usuario_id	BIGINT	FOREIGN KEY	Usuario dueño de la ranura
fecha_guardado	DATETIME	DEFAULT NOW	Estampa temporal de último guardado
ronda_actual	INT UNSIGNED	NOT NULL (1)	Ronda en la que se quedó el combate
modo_juego	ENUM	manual/automatico	Modo activo del juego
terminada	BOOLEAN	DEFAULT FALSE	Flag que indica si el trial concluyó
vacía	BOOLEAN	DEFAULT TRUE	Determina si el slot está activo o vacío

Tabla: PERSONAJE_PARTIDA (Estado del Personaje)

Campo	Tipo de Datos	Restricción	Descripción
id_personaje	INT	AUTO_INCREMENT PK	Clave primaria del personaje
id_ranura	INT	FOREIGN KEY	Ranura vinculada (ON DELETE CASCADE)
usuario_id	BIGINT	FOREIGN KEY	Usuario propietario (ON DELETE CASCADE)
clase_personaje	VARCHAR(100)	NOT NULL	Clase exacta en Java por Reflection
nombre	VARCHAR(100)	NOT NULL	Nombre del personaje (ej. Leon Kennedy)
vida_actual	INT UNSIGNED	NOT NULL	Vida en el momento del guardado
vida_max	INT UNSIGNED	NOT NULL	Vida máxima del personaje
defensa_base	INT UNSIGNED	NOT NULL	Defensa inherente
bando	ENUM	survi / killer	Facción del personaje

Tabla: PERK_EQUIPADA (Habilidades en Partida)

Campo	Tipo de Datos	Restricción	Descripción
id_perk_equipado	INT	AUTO_INCREMENT PK	Clave primaria
id_personaje	INT	FOREIGN KEY Personaje portador (ON DELETE CASCADE)	
clase_perk	VARCHAR(100)	NOT NULL	Clase concreta de la perk
nombre	VARCHAR(100)	NOT NULL	Nombre descriptivo de la habilidad
usos_restantes	INT UNSIGNED	NOT NULL Usos de habilidad que le quedan al survivor/killer	

CAPÍTULO 4: DICCIONARIO DE CONSULTAS Y QUERIES DEL SISTEMA

4.1 PERSISTENCIA HÍBRIDA: JDBC Y JPA

Para lograr el máximo control transaccional durante el guardado y carga de la partida, el sistema utiliza consultas nativas ejecutadas por JDBC de forma atómica. Esto asegura que si el guardado de una perk o arma falla, toda la transacción se cancela y se revierte en la base de datos a su estado anterior mediante un ROLLBACK automático.

QUERY 1: LIMPIEZA ATÓMICA DE DATOS ANTERIORES DE LA RANURA

Antes de insertar el nuevo estado de los personajes, se limpian todos los registros de la ranura para el usuario activo. El motor SQL se encarga de borrar en cascada las armas, perks y estados correspondientes.

```
DELETE FROM PERSONAJE_PARTIDA WHERE id_ranura = ? AND usuario_id = ?;
```

QUERY 2: ACTUALIZACIÓN DE METADATOS DE LA RANURA

Actualiza los campos de control de la ranura de guardado asignando la ronda, modo, bando y dificultad elegida.

```
UPDATE RANURA SET
  ronda_actual = ?, modo_juego = ?, bando = ?, dificultad = ?,
  terminada = ?, vacia = FALSE
WHERE id_ranura = ? AND usuario_id = ?;
```

QUERY 3: INSERCIÓN POR LOTES E HISTÓRICO ABSOLUTO DE PERSONAJES

Cuando el trial termina, se guardan los resultados finales en el histórico absoluto de manera optimizada usando Batch Updates para insertar decenas de registros en un solo viaje de red.

```
INSERT INTO HISTORICO_PERSONAJES (id_hist_partida, nombre, bando, vida_final) VALUES (?,
?, ?, ?);
```

QUERY 4: CONSULTA AVANZADA DE LOGROS DESBLOQUEADOS POR USUARIO

Para renderizar en Next.js el listado completo de logros marcando dinámicamente cuáles han sido conseguidos por el usuario autenticado, realizamos un LEFT JOIN eficiente de la tabla maestra con la tabla de relación intermedia:

```
SELECT l.id_logro, l.nombre, l.descripcion,
  CASE WHEN ul.usuario_id IS NOT NULL THEN TRUE ELSE FALSE END as conseguido
FROM LOGROS l
LEFT JOIN USUARIOS_LOGROS ul
  ON l.id_logro = ul.id_logro AND ul.usuario_id = ?;
```

CAPÍTULO 5: SEGURIDAD DE TOKENS Y OPTIMIZACIÓN DEL INP

5.1 SEGURIDAD AVANZADA: COOKIES HTTPONLY Y HANDSHAKE JWT

En versiones anteriores del proyecto, el token JWT de autenticación se almacenaba directamente en el 'localStorage' del navegador. Esta práctica expone el token a scripts maliciosos que puedan inyectarse (ataques XSS). Para erradicar esta vulnerabilidad, hemos rediseñado el handshake de seguridad:

- Al autenticarse con éxito en `/api/auth/login`, el backend Spring Boot inyecta el token en una cookie con la directiva HttpOnly.
- Esta cookie viaja de forma automática y transparente en la cabecera 'Cookie' de todas las peticiones fetch relativas.
- El frontend cuenta con un middleware de Next.js que intercepta la cookie, verifica su autenticidad y restringe el acceso al dashboard a nivel de servidor (Server-Side Rendering), evitando parpadeos de interfaz y accesos no autorizados.

5.2 OPTIMIZACIÓN DE RENDIMIENTO FRONTEND Y AJUSTES DEL INP

El Interaction to Next Paint (INP) es una métrica de Core Web Vitals crucial que mide la latencia de respuesta de la interfaz ante las interacciones del usuario. Durante la fase de auditoría del frontend, identificamos que el INP era excesivamente elevado debido a dos cuellos de botella críticos del diseño CSS que sobrecargaban la CPU de renderizado:

1. ELIMINACIÓN DE LA PROPIEDAD 'bg-fixed':

Tener una imagen de fondo de alta resolución fija (`background-attachment: fixed`) mientras el usuario hace scroll obliga al navegador a re-calcular los píxeles y re-pintar la pantalla completa en cada frame. Al eliminar esta propiedad y sustituirla por un contenedor de fondo absoluto estático, la carga de re-pintado se redujo a cero, logrando un scroll ultra-suave a 60fps.

2. PROMOCIÓN DE CAPAS ACELERADAS POR GPU (`will-change`):

En elementos altamente interactivos como la máquina de escribir de los diálogos, los botones dinámicos y la selección de personajes, añadimos la propiedad CSS `will-change: transform, opacity`. Esto le avisa al navegador con antelación de qué elementos van a cambiar, permitiéndole promoverlos a una capa independiente de renderizado gestionada directamente por la GPU en lugar de sobrecargar el hilo principal de la CPU. El INP se redujo drásticamente de 180ms a menos de 40ms.

CAPÍTULO 6: MANUAL COMPLETO DE DESPLIEGUE EN AWS

6.1 FASE 1: ARQUITECTURA DE RED Y SEGURIDAD (VPC)

Para un despliegue seguro, creamos una VPC propia con dos subredes públicas y dos privadas.

- Creamos un Grupo de Seguridad para la máquina de aplicaciones (EC2) que permita entrada exclusivamente en los puertos:

- * Puerto 22 (SSH): Restringido únicamente a nuestra IP de administración.

- * Puerto 80 (HTTP) y Puerto 443 (HTTPS): Abierto a todo el público para cargar la web.

- Creamos un Grupo de Seguridad para la base de datos (RDS MariaDB) que restrinja el acceso al puerto 3306, permitiendo peticiones únicamente si provienen del Grupo de Seguridad de la instancia EC2.

6.2 FASE 2: INSTANCIAR BASE DE DATOS (RDS MARIADB)

1. En la consola de AWS RDS, lanzamos una base de datos MariaDB (versión 10.11 o similar) en la capa gratuita (t3.micro / single AZ).

2. Configuramos las credenciales maestras y deshabilitamos el acceso público.

3. Una vez activa, nos conectamos desde nuestra máquina local (mediante un túnel SSH o temporalmente habilitando el puerto) y ejecutamos el script de inicialización ``backend/script_bbdd.sql`` para poblar las tablas, ranuras vacías y logros por defecto.

6.3 FASE 3: ALOJAMIENTO EN SERVIDOR EC2

1. Lanzamos una instancia EC2 con sistema operativo Ubuntu Server LTS (capa gratuita).

2. Nos conectamos por SSH e instalamos los requisitos previos esenciales:

- JDK 17: ``sudo apt install openjdk-17-jdk -y``

- Node.js 18+ y npm: ``curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash - && sudo apt install nodejs -y``

- Gestor de procesos PM2 global: ``sudo npm install pm2 -g``

3. CLONADO Y CONFIGURACIÓN DEL BACKEND:

- Clonamos el repositorio del proyecto en ``/var/www/dbd-rpg``.

- Editamos el archivo ``backend/src/main/resources/application.properties`` para apuntar a la IP/Endpoint de RDS:

```
`spring.datasource.url=jdbc:mariadb://[ENDPOINT_DE_RDS]:3306/dbd_rpg`
```

- Compilamos la aplicación de Spring Boot: ``./mvnw clean package -DskipTests``.

- Levantamos el backend en segundo plano como servicio de systemd o vía PM2 para garantizar auto-arranque:

```
`pm2 start "java -jar target/rpg-0.0.1-SNAPSHOT.jar" --name dbd-backend`
```

4. COMPILACIÓN Y ARRANQUE DEL FRONTEND:

- En ``frontend/next.config.ts``, aseguramos que las llamadas relativas ``/api`` apunten internamente a ``http://localhost:8080``.

- Instalamos dependencias y compilamos la versión de producción:
``pnpm install && pnpm run build``
- Iniciamos el servidor de Next.js en el puerto 3000 de forma permanente:
``pm2 start "pnpm run start" --name dbd-frontend``

6.4 FASE 4: PROXY INVERSO CON NGINX Y ENCRIPCIÓN SSL

1. Instalamos Nginx: ``sudo apt install nginx -y``
 2. Creamos el archivo de configuración del sitio en ``/etc/nginx/sites-available/dbd-rpg``:
 - Configuramos la raíz para redirigir las peticiones HTTP normales al frontend en el puerto 3000.
 - Mapeamos ``/api/*`` y ``/ws/*`` directamente hacia el backend en ``http://localhost:8080`` de forma transparente.
 3. Habilitamos el sitio y reiniciamos Nginx:
``sudo ln -s /etc/nginx/sites-available/dbd-rpg /etc/nginx/sites-enabled/``
``sudo systemctl restart nginx``
 4. ENCRIPCIÓN SSL CON LET'S ENCRYPT:
 - Instalamos Certbot: ``sudo apt install certbot python3-certbot-nginx -y``
 - Obtenemos el certificado SSL automático configurando automáticamente HTTPS:
``sudo certbot --nginx -d [TU_DOMINIO.com]``
- ¡Tu RPG de Dead by Daylight estará en línea y seguro con calificación de 10!